JAVA INTERVIEW PREPARATION

CHAPTER 50

JPA ASSOCIATIONS AND
FETCH PLANS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

JPA Associations and Fetch Plans: Lazy Loading, Join Fetching, and the N+1 Problem

Senior / Lead Java Developer Reading Chapter

JPA associations make relational links look like object navigation. That convenience can hide when SQL executes, how many rows are multiplied, and whether data remains available after the persistence context closes. Senior JPA design treats fetching as a use-case decision rather than a permanent annotation choice.

Association Ownership Is About the Foreign Key

In a bidirectional relationship, the owning side controls the database association. Ownership does not mean business ownership or aggregate ownership.

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "order_id", nullable = false)
private Order order;                // owning side: contains FK

@OneToMany(mappedBy = "order")
private List<OrderLine> lines;      // inverse side
```

Changing only the inverse collection may leave the foreign key unchanged. Helper methods should keep both Java sides consistent:

```
public void addLine(OrderLine line) {
    lines.add(line);
    line.attachTo(this);
}
```

Database constraints remain final authority. A Java collection cannot enforce referential integrity across every writer.

To-One and To-Many Mapping Choices

`@ManyToOne` maps many source rows to one target and typically stores the foreign key. `@OneToMany` represents the inverse collection. `@OneToOne` requires a unique foreign key or shared primary key to be truly one-to-one. `@ManyToMany` uses a join table.

Many-to-many is often too weak for a real domain relationship. If membership has status, role, order, timestamps, audit, or lifecycle, model the join as an entity such as `ProjectMembership`.

```
User <-- ProjectMembership --> Project
         role, joinedAt, status
```

This provides identity, constraints, optimistic versioning, and clearer update semantics.

FetchType Is a Default, Not a Query Plan

JPA defaults to EAGER for many-to-one and one-to-one and LAZY for one-to-many and many-to-many. Defaults are contractual hints and provider behavior can require proxies or enhancement. EAGER does not guarantee one SQL join; the provider may issue secondary selects. LAZY does not guarantee the association stays unloaded when another fetch plan requests it.

Mapping every association EAGER is not a fix. Different endpoints need different graphs. A list page may need order summary only, while a detail page needs lines and shipping address. One static eager graph either over-fetches common queries or remains insufficient for complex ones.

Prefer lazy mappings as a safe baseline where provider support is reliable, then request exact data for each use case.

How N+1 Appears

N+1 means one query loads N parent rows, then navigation triggers another query for each parent.

```
List<Order> orders = repository.findRecent(); // 1 query
for (Order order : orders) {
    render(order.getCustomer().getName());    // up to N queries
}
```

```
select ... from orders limit 100
select ... from customer where id=?  -- repeated
select ... from customer where id=?
...
```

The code appears linear and tests with three rows look harmless. Production latency grows with result cardinality and round trips. First-level and second-level cache hits can mask N+1 in one environment but not another.

N+1 also occurs in serialization, logging, mapping, validation, `toString`, equality, or template rendering. Measure SQL for the full request boundary, not only repository method execution.

Join Fetch

JPQL fetch join retrieves an association as part of the query result graph.

```
select distinct o
from Order o
join fetch o.customer
left join fetch o.lines
where o.id = :id
```

It is effective for one known aggregate or a to-one association. Fetching a collection multiplies the root row once per child. JPA deduplicates root entity identity, but the database still transfers every joined row.

Joining two to-many collections can produce a Cartesian multiplication. Ten lines and five adjustments can yield fifty result rows for one order. Hibernate can also reject simultaneous fetching of certain bag collections. Split queries, batch fetching, or projections may be better.

Pagination and Collection Fetch Joins

Database pagination operates on joined rows, while the application wants root entities. A collection fetch join with `LIMIT 20` can return fewer than twenty unique orders or force in-memory pagination, depending on provider behavior.

Use a two-step pattern:

```
1. page root IDs with deterministic order
2. fetch required graph for those IDs
3. restore page order if necessary
```

Alternatively use a DTO projection that returns the exact row shape, or batch-fetch associations after loading the page. Always inspect generated SQL and warnings.

Entity Graphs

Entity graphs define requested attributes without embedding the plan in JPQL.

```
@EntityGraph(attributePaths = {"customer", "shippingAddress"})
Optional<Order> findDetailedById(UUID id);
```

Jakarta Persistence distinguishes fetch graph and load graph semantics. A fetch graph treats specified attributes as eager and unspecified attributes as lazy; a load graph treats specified attributes as eager while unspecified attributes follow mapping defaults.

Subgraphs describe nested associations. Entity graphs improve reuse and keep query predicates separate from graph selection, but they do not eliminate row multiplication or guarantee one SQL shape. Verify the provider's generated plan.

DTO Projections

A read use case often does not need managed entities.

```
select new com.example.OrderSummary(
    o.id, c.displayName, o.status, o.total, o.createdAt)
from Order o
join o.customer c
where o.tenantId = :tenant
order by o.createdAt desc
```

DTO projections select explicit columns, avoid accidental lazy navigation, reduce dirty-checking overhead, and create a stable API mapping boundary. They are particularly strong for list, report, and dashboard queries.

Projection trades automatic aggregate navigation for explicit query ownership. When a command must enforce entity behavior and persist changes, load the aggregate with the associations required for that command.

Batch Fetching

Batch fetching groups several pending lazy loads into an `IN` query.

```
without batching: customer id 1, then 2, then 3 ...
with batching:    select ... where id in (1,2,3,...)
```

Hibernate supports default and mapping-specific batch sizes. Batch fetching reduces round trips but still relies on access timing and persistence-context state. It is a mitigation, not an explicit query contract.

Choose a batch size compatible with database parameter limits, row width, and expected working set. Extremely large `IN` lists increase parsing and planning cost.

Subselect Fetching

Provider-specific subselect fetching can load collections for all parents returned by an earlier query using that query's IDs. It can work well when a page of parents will all have the same collection accessed.

It is sensitive to persistence-context contents and can unexpectedly fetch more data than the immediate code suggests. Treat it as a measured provider optimization and test memory and SQL behavior.

To-One Proxies and Optional Associations

Lazy to-one loading may use a proxy containing target type and identifier, or bytecode-enhanced unloaded state. Calling ordinary business getters can initialize it. Identifier access may avoid initialization in provider-specific circumstances, but code should not build correctness around undocumented proxy behavior.

Optional associations are harder to proxy when absence cannot be known without querying. Shared primary-key one-to-one mappings and `optional=false` provide different information to the provider than a nullable foreign key. Inspect SQL for the exact mapping and enhancement configuration.

Proxy runtime class also affects equality, serialization, and debugging. Avoid strict assumptions that every association is the concrete entity class. Do not call `toString`, Lombok-generated equality, or debugger expansion across lazy graphs without considering initialization.

DISTINCT and Database Work

JPQL `distinct` can remove duplicate root entity references caused by a collection join. It does not undo the database row multiplication required to materialize children. Provider versions differ in whether DISTINCT is passed to SQL or handled in memory.

```
1 order x 100 lines -> database still emits 100 joined rows
JPA identity map -> one Order instance with 100 line instances
```

Use DISTINCT for correct root result semantics when needed, not as a performance cure. Inspect the execution plan because database DISTINCT may add sorting or hashing on an already wide joined row.

Fetch Plans for Commands Versus Queries

A command should load enough aggregate state to decide and enforce its transition. Fetching less can trigger hidden lazy SQL during domain behavior; fetching every historical association can make the command unbounded.

A query endpoint often needs a denormalized projection rather than the aggregate. Separating command loading from read projections is a practical form of CQRS without requiring separate infrastructure.

```
CancelOrder command -> Order + current cancellation-relevant state
OrderHistory view   -> DTO projection across order, events, and actor display data
```

Name repository methods after use cases, such as `findForCancellation` or `findSummaryPage`, rather than exposing generic graphs whose callers navigate unpredictably.

Fetching and Authorization

Tenant and ownership predicates should be part of the root query, not a check after loading an unrestricted entity. Fetch joins and entity graphs must preserve those predicates. A cached or lazy association must not cross tenant boundaries because its target was loaded by ID alone.

Repository tests should assert generated SQL or result isolation for multi-tenant mappings. Filters can help but require correct activation on every context, including background jobs. Database row-level security can add defense, but JPA still needs explicit domain authorization.

Streaming and Large Graphs

Streaming root rows does not make an eagerly joined large collection streamable in constant memory. The persistence context retains managed identities and collections, and JDBC drivers may buffer results unless fetch-size and transaction behavior are configured.

For exports, prefer DTO/scalar streaming with bounded fetch size, keyset chunks, or database-native export. Clear managed state between chunks and avoid lazy per-row calls. Define who closes the stream and transaction when exceptions or client disconnects occur.

LazyInitializationException

Lazy loading requires an active persistence context and provider connection path. Accessing an unloaded association after the transaction closes can throw `LazyInitializationException`.

```
service transaction closes
  -> detached Order returned
  -> controller serializer accesses order.lines
  -> no active context: failure
```

The fix is not automatically Open Session in View or making everything eager. Define the response graph inside the use-case transaction and map to a DTO while data is intentionally available.

Open Session in View moves SQL into controllers or serialization, hides transaction boundaries, and can produce many queries after business logic completes. It also makes performance depend on which fields a serializer touches.

Collection Semantics

Choose `Set`, `List`, ordered list, bag, or map from domain semantics and mapping cost. A Set requires stable equality and hashing. An ordered list may persist an order column and update many positions after insertion. A bag allows duplicates and has different fetch behavior.

Do not use Set solely to hide duplicate roots caused by a join. That may mask SQL multiplication while losing meaningful order. Make uniqueness and order explicit in database constraints and mappings.

Orphan Removal and Cascades

`orphanRemoval=true` schedules a child for deletion when removed from the owning collection. It fits children whose lifecycle belongs exclusively to the aggregate. It is dangerous for shared or independently meaningful rows.

Cascades propagate lifecycle operations, not fetch behavior. `CascadeType.ALL` does not make associations eager and should not be used by habit. Test SQL when replacing collections; naive mapping from a DTO can delete and reinsert every child.

Filtering Associations

An entity association represents a mapped relationship, not every possible filtered view. A collection such as `order.lines` should not change meaning based on a request-specific status filter.

Use a query or projection for filtered subsets. Provider filters and SQL restrictions can be appropriate for stable cross-cutting rules such as tenant or soft-delete visibility, but they affect caches and every access path and require rigorous testing.

Query Count and Row Count

Reducing query count from 101 to one is not automatically an improvement if the one query transfers a million multiplied rows. Measure:

- Number of SQL statements.
- Rows returned and bytes transferred.

- Database execution and lock time.
- Serialization and object allocation.
- Persistence-context entity and collection count.
- End-to-end latency under realistic cardinality.

The target is minimal total work with predictable behavior, not a slogan such as "one query good."

Testing Fetch Plans

Create representative cardinalities: orders with zero, typical, and extreme child counts. Assert SQL count for critical paths using datasource instrumentation or provider statistics. Verify pagination returns the requested number of unique roots.

Close or clear the persistence context before tests intended to reveal lazy access. A warm first-level cache can hide missing fetches. Use the production database because join plans and parameter limits differ.

Contract-test serialization from DTOs. Entity serialization tests can pass only because a test transaction remains open, hiding the production boundary.

Production Diagnosis

Trace repository operations and collect normalized SQL/query counts without logging sensitive values. A sudden latency rise after adding one response field often signals lazy navigation or join multiplication.

Use database plans to distinguish N+1 round trips from one slow join. Inspect heap when a fetch graph materializes a massive object tree. Add query budgets and payload limits to prevent one request from loading unbounded associations.

Common Senior-Level Traps

Traps include setting all mappings EAGER, assuming EAGER means one join, collection-fetching multiple bags, paginating a collection fetch join, enabling Open Session in View to hide boundary errors, and treating batch fetching as a complete query design.

Another trap is focusing only on query count. Row multiplication, managed-object allocation, and serialization can dominate after N+1 is removed.

A Strong Interview Answer

> **I treat fetch shape as a use-case contract. Associations are usually lazy by baseline, then commands load the aggregate they must modify and reads use join fetch, entity graphs, batch loading, or DTO projection according to cardinality. I detect N+1 across the complete request because mapping and serialization can trigger it. Collection joins multiply rows and interact badly with pagination, so I page root IDs first or use projections. I do not fix LazyInitializationException with global eager loading or an open session; I map intentional data inside the transaction. Tests assert queries, rows, unique page roots, and extreme child counts against the real database.**

Interview-Ready Summary

JPA associations describe relationships, while fetch plans describe what one use case needs now. Static eager mappings cannot serve every workflow. N+1, join multiplication, pagination, and lazy boundaries must be evaluated together.

Join fetch, entity graphs, DTO projections, batch fetching, and split queries are complementary tools. The best plan minimizes total database, network, heap, and serialization work while keeping transaction and API boundaries explicit.

Revision Notes

- Association ownership means control of the foreign key mapping.
- Keep both sides of bidirectional Java relationships consistent.
- FetchType is a default hint, not a complete query plan.
- EAGER does not guarantee one joined SQL statement.
- N+1 is one root query plus repeated association queries.
- Mapping, logging, and serialization can trigger lazy SQL.
- Fetch joins solve selected N+1 cases but multiply collection rows.
- Multiple to-many joins can create Cartesian multiplication.
- Collection fetch joins and database pagination conflict.
- Page root IDs first when a graph must be loaded afterward.
- Fetch graph and load graph have different unspecified-attribute semantics.
- DTO projections are strong for read-only use-case views.
- Batch fetching reduces round trips but remains access-pattern dependent.
- LazyInitializationException signals an undefined fetch boundary.
- Open Session in View hides SQL beyond the service transaction.
- Collection type affects equality, order, and update behavior.
- Orphan removal fits exclusively owned child lifecycle.
- Cascades do not control fetching.
- Measure queries, rows, bytes, objects, and end-to-end latency.
- Test typical and extreme association cardinalities.